

Microservices Deployment Architecture with Kubernetes Clusters

Scalability and resilience are two of the most important reasons to move from monoliths to microservices. The Kubernetes platform offers both while orchestrating containers. In this Part 9 of the series, reference architecture for a user management system is presented and demonstrated around Kubernetes. This architecture includes Spring Boot microservices, Apache Kafka and MySQL.



Look at any e-commerce business. It flourishes during the weekends and on special days. The business is normally low till noon and peaks in the evenings. So the systems that back such e-commerce, banking and government services, etc, experience different loads at different points of time, and need to be scaled up or down as automatically as possible. Such systems require appropriate deployment architecture and orchestration tooling.

In the previous part of this series of articles, we have seen how docker-compose is useful in deploying multiple containerised services all at once, on a single machine. Though docker-compose is good enough for container

deployment, it falls short when it comes to container orchestration. It cannot track the containers and maintain the stability of the infrastructure. That's where Kubernetes comes to our rescue.

Kubernetes can deploy containerised services not just on one machine but also on a cluster of any number of machines. It can deploy multiple instances of the same service across the cluster. Kubernetes keeps track of each of the deployed containers. And in case of crashes, it manages the scalability levels by automatically bringing up replacement containers without any manual intervention.

Let us architect the UMS (user management system) deployment with the help of Kubernetes.